

ABE CPU 演化优化：并行结构与向量化

目 录

1 引言：ABE 的并行结构还能挖多少	2
2 ABE 的并行结构现状	2
2.1 MPI 已用，OpenMP 未启用	2
2.2 关键源文件一览	2
3 并行结构优化：不是简单加 OpenMP	3
3.1 表面串行但内部可并行的机会	3
3.2 已并行但粒度或调度不适合	3
3.3 OpenMP fork-join 开销	3
3.4 重新平衡 MPI rank 与 OpenMP thread	3
3.5 检查多线程竞争	3
3.6 绑核与 NUMA 设置避免迁移波动	3
4 NUMA 拓扑与绑定	3
4.1 用工具看拓扑	3
4.2 绑核方式	4
4.3 Kunpeng 920B 上混合并行受 NUMA 影响显著	4
5 计算 kernel 优化：让 bssn_rhs 跑得更快	4
5.1 帮助编译器自动向量化	4
5.2 用 lscpu 确认指令集	4
5.3 热点循环尝试 AArch64 SIMD intrinsic	5
5.4 简单并行循环用 OpenMP	5
6 通信优化：rank 数上去后通信会成瓶颈	5
6.1 ghost zone 交换通信量	5
6.2 是否过多同步	5
6.3 合并小消息	5
6.4 重叠通信与计算	5
6.5 rank 间负载均衡	5
7 AMSS_NCKU_Input.py 里的关键参数	6
8 本章你将学会	6
9 要点速查	6
10 小结	7

1 引言：ABE 的并行结构还能挖多少

在 Lab4 里，task1 的目标是把 **TwoPunctureABE + ABE** 在 **Kunpeng 920B**（一颗 ARM 处理器）上的端到端时间压下来。ABE 是 **BSSN** 形式下的 CPU 演化代码，baseline 只用了 **MPI**（消息传递接口）做进程级并行，并没有启用 **OpenMP**（开放式多处理）。

这件事乍看不严重，其实留下了相当大的优化空间。首先，CMake 默认不开 OpenMP，OMP_threads 这个参数当前只是个环境变量提示，没有真正生效的 OpenMP 指令。其次，演化代码里许多热点循环要么靠编译器自动向量化，要么靠手写 SIMD，baseline 都没认真做。再次，进程数、绑核、NUMA（非一致性内存访问）这些和并行结构强相关的配置，baseline 也没调过。

本讲我们就围绕 ABE 的并行结构与向量化展开：先看现状，再看怎么动 MPI 与 OpenMP 的混合并行，怎么用 NUMA 工具绑核，怎么把 bssn_rhs.f90 里的热点循环向量化，最后看通信瓶颈怎么分析。

2 ABE 的并行结构现状

我们先把现状摸清楚，再谈优化。

2.1 MPI 已用，OpenMP 未启用

ABE baseline 已经用 MPI 把网格划分到多个进程上，每个 **rank**（MPI 进程）负责一部分网格。但 CMake 没有开 OpenMP，也就是说，即使源码里偶尔有 `#pragma omp` 之类的指令，编译器也认不出来。OMP_threads 在 AMSS_NCKU_Input.py 里只是设了一个环境变量，它不会自己产生并行。

直觉 你可以把 MPI 想成“分给几个工人不同的房间”，OpenMP 想成“每个房间里有几个工人一起干”。现在每个房间里实际只有一个工人在动，环境变量喊“四个工人”也没用，因为根本没开门。

2.2 关键源文件一览

文件	职责	优化关注点
bssn_rhs.f90	BSSN 方程右端项	热点循环, 向量化
diff_new.f90	有限差分	模板访问, 向量化
lopsidediff.f90	单侧差分	边界处理
kodiss.f90	Kreiss-Oliger 耗散	与差分类似
rungekutta4_rout.f90	RK4 时间推进	多步串行, 可重叠
prolongrestrict_cell.f90	网格层级间延长与限制	AMR 开销
Parallel.C	MPI 通信封装	通信量, 同步
MPatch.C	patch 管理	任务划分, 负载均衡

后续优化主要围绕 bssn_rhs.f90 及它调用的差分与耗散函数，再加上 Parallel.C 与 MPatch.C 里的通信与划分逻辑。

3 并行结构优化：不是简单加 OpenMP

很多人第一次做 ABE 优化，会想“在循环前加一行 `#pragma omp parallel for` 就完事”。这常常是错的。我们要分几种情况看。

3.1 表面串行但内部可并行的机会

有些代码看起来是一层串行循环，但循环体里在遍历多个 **block**（块）、**patch**（块网格）、**detector**（探测器）或网格层级。这些“内层”对象之间往往没有数据依赖，是可以并行的。profiler 里看热点，再去源码里找这些机会，比盲目加指令靠谱。

3.2 已并行但粒度或调度不适合

有些循环已经并行了，但任务粒度太小，fork-join 开销大于收益；或者调度策略（static/dynamic/guided）不适合当前数据分布；或者数据划分让某些线程频繁访问远端内存。这些都要在当前平台上重新平衡。

3.3 OpenMP fork-join 开销

Fork-join（分叉与合并）是 OpenMP 的固有开销。并行区域太小，启动线程的开销可能比省下的计算还多。判断标准是看并行区域内的总工作量是否远大于启动开销，经验上每个线程至少要有几百微秒的工作。

3.4 重新平衡 MPI rank 与 OpenMP thread

在固定资源下，rank 多了，每个 rank 的内存与通信开销会涨；rank 少了，每个 rank 内的并行度不够。加 OpenMP 后，可以让 rank 数减少，每个 rank 内用多线程填充。典型的混合模式是“每 NUMA 节点一个 rank，rank 内多线程”。

3.5 检查多线程竞争

多个线程如果频繁写同一个缓存行，会产生 **false sharing**（假共享）；如果竞争同一个共享缓冲区，会产生锁等待。这些在 profiler 里表现为缓存命中差或同步等待长。

提示：发现并行后反而变慢，第一反应应该是怀疑 *false sharing* 或远端内存访问，而不是“OpenMP 没用”。

3.6 绑核与 NUMA 设置避免迁移波动

不绑核的话，操作系统可能把线程在核之间迁移，导致缓存失效与远端内存访问。绑核能让性能稳定，便于对比优化效果。

直觉 把线程绑在核上，像让每个工人固定在一个工位，工具与材料都顺手；不绑的话，工人时不时换工位，每次都要重新找东西。

4 NUMA 拓扑与绑定

Kunpeng 920B 是一颗多核 ARM 处理器，有明显的 NUMA 层级。理解 NUMA 拓扑是绑核的前提。

4.1 用工具看拓扑

1. `lscpu` 看总体架构、核数、缓存层级、NUMA 节点数；

2. numactl --hardware 看 NUMA 节点与内存分布；
3. hwloc-ls 用图形或树形显示硬件拓扑，最直观。

提示：先跑这三条命令，把当前机器的拓扑截图存下来，再决定 rank 与线程怎么摆。

4.2 绑核方式

1. mpirun 自带的绑核参数，如 --bind-to、--map-by；
2. OpenMP 的 OMP_PROC_BIND 与 OMP_PLACES，控制线程是否绑定与绑定到哪些核；
3. numactl --cpunodebind 与 numactl --membind，把进程绑定到某 NUMA 节点的核与内存。

例 假设你的机器有 2 个 NUMA 节点，每节点 24 核，共 48 核。你想用 2 个 MPI rank，每 rank 12 个 OpenMP 线程，且 rank 与线程都绑在自己 NUMA 节点上。

```
mpirun -np 2 --map-by ppr:1:node:pe=12 \  
      --bind-to core --report-bindings \  
      ./ABE.exe  
export OMP_PROC_BIND=close  
export OMP_PLACES=cores
```

rank 0 绑在 NUMA 0 的核上，rank 1 绑在 NUMA 1 的核上，每个 rank 内 12 个线程紧密排列。这样既避免了跨 NUMA 远端内存访问，也避免了核间迁移波动。

4.3 Kunpeng 920B 上混合并行受 NUMA 影响显著

在 Kunpeng 920B 上，MPI+OpenMP 混合并行对 NUMA 极其敏感。如果 rank 跨 NUMA 节点，或者线程访问的内存不在本节点，远端访问延迟会吃掉大半性能。所以“绑核 + 本地内存”是稳定性能的前提。

5 计算 kernel 优化：让 bssn_rhs 跑得更快

并行结构搭好之后，下一步是把每个线程算得更快。重点放在 bssn_rhs.f90 及它调用的差分与耗散函数上。

5.1 帮助编译器自动向量化

编译器的自动向量化能识别简单循环，但有几样东西会挡它的路：

1. 循环体内的条件分支，特别是数据相关的分支；
2. 循环携带依赖，即下一次迭代的输入依赖上一次迭代的输出；
3. 不连续的内存访问，比如跨 stride 取数；
4. 别名问题，即编译器无法确认两个指针是否指向同一块内存。

我们能做到的是把这些“路障”清掉：把分支挪出循环、用 restrict 或 Fortran 的 intent(in) 帮编译器排除别名、让数据布局连续。

5.2 用 lscpu 确认指令集

Kunpeng 920B 是 ARM 架构，支持 AArch64 指令集。进一步要看它支持 NEON 还是 SVE，这决定了你能用的 SIMD 指令宽度。

```
lscpu | Select-String -Pattern "Architecture|Flags|Model name"
```

或者在 Linux 下直接 `cat /proc/cpuinfo` 看 Features 行。

5.3 热点循环尝试 AArch64 SIMD intrinsic

当自动向量化不够时，可以手写 **SIMD intrinsic**（单指令多数据内联函数）。ARM 下对应的是 NEON 或 SVE 的 intrinsic。这是手活，要先 profile 出真正的热点，再针对那一两个循环手写，别一上来就全局改。

5.4 简单并行循环用 OpenMP

对于明显的独立循环，加 `!$omp parallel do`（Fortran 形式）即可。注意变量作用域：默认共享的变量如果不该共享，要用 `private` 或 `firstprivate` 显式声明。

直觉 自动向量化是“四个数一起算”，SIMD intrinsic 是“你亲手把四个数塞进宽寄存器”，OpenMP 是“多个线程同时搬”。三者不互斥，常常一起用。

6 通信优化：rank 数上去后通信会成瓶颈

演化代码的通信主要来自 **ghost zone**（鬼区）交换：每个 rank 在自己网格边界上需要邻居的部分数据来算差分。rank 数越多，边界相对体积越大，通信占比越高。

6.1 ghost zone 交换通信量

设每个 rank 持有 n^3 个网格点，ghost 区厚度为 g ，则每个面要收 n^2g 个数。一个立方体有 6 个面，总通信量近似正比于 $6n^2g$ ，而计算量正比于 n^3 。当 n 减小（rank 增多），通信占比按 $1/n$ 增长。

6.2 是否过多同步

每次演化步里，差分之前都要等 ghost 区到位。如果同步点太多，rank 之间会频繁互相等。检查代码里是否有不必要的 `MPI_Barrier`，能否合并。

6.3 合并小消息

MPI 对每条消息都有固定开销，小消息多反而慢。把同一方向的多块小数据合并成一条消息，能显著降低延迟占比。

6.4 重叠通信与计算

可以把 ghost 区交换与内部计算重叠：先发起非阻塞通信 `MPI_Isend/MPI_Irecv`，算完内部再 `MPI_Wait`。这样通信时间被计算时间掩盖。

6.5 rank 间负载均衡

如果某些 rank 分到的网格复杂、AMR 层级多，它们会早完成去等其他 rank。任务划分要尽量让每个 rank 的工作量接近，否则最慢的 rank 拖死整体。

例 假设网格分成 4 个 rank，每 rank 持有 $64 \times 64 \times 64$ 个点，ghost 厚度 $g = 3$ 。

每 rank 一个面的 ghost 量是 $64 \times 64 \times 3 = 12288$ 个数。立方体有 6 个面，理想情况下每步总收发约 $6 \times 12288 = 73728$ 个数（实际按邻居关系算）。

每个 rank 的计算量是 $64^3 = 262144$ 个点的 RHS。通信量与计算量之比约为 $73728/262144 \approx 0.28$ 。

如果 rank 数加到 8，每 rank 只剩 $32 \times 64 \times 64 = 131072$ 个点，每面 ghost 仍 $64 \times 64 \times 3 = 12288$ ，比值升到 $73728/131072 \approx 0.56$ ，通信占比翻倍。这就是“rank 越多通信越贵”的直观来源。

7 AMSS_NCKU_Input.py 里的关键参数

配置文件里有几个参数直接影响并行结构：

1. MPI_processes: MPI rank 数，直接决定进程级并行度；
2. OMP_threads: OpenMP 线程数，但要注意，它当前只是环境变量提示，要让 OpenMP 真正生效，必须先在源码里加 OpenMP 指令，再在 CMake 里启用 OpenMP 支持，否则这个值喊再大声也没用。

坑点：很多人调大 OMP_threads 期望加速，结果没变快，就是因为 CMake 没开 OpenMP。先确认 CMake 里 `find_package(OpenMP)` 与编译选项都到位，再去调这个值。

直觉 把 MPI_processes 当成“开几个房间”，OMP_threads 当成“每房间几个工人”。两者乘积不能超过总核数，否则互相抢，反而变慢。

8 本章你将学会

1. 分析 ABE 的并行结构现状，理解 MPI 已用、OpenMP 未启用的含义；
2. 设计 MPI+OpenMP 混行并行，重新平衡 rank 数与线程数；
3. 用 `lscpu`、`numactl --hardware`、`hwloc-ls` 看 NUMA 拓扑，并用 `mpirun` 绑核参数、`OMP_PROC_BIND/OMP_PLACES`、`numactl` 绑核；
4. 优化 `bssn_rhs.f90` 及调用函数的向量化，从清路障到自动向量化再到 AArch64 SIMD intrinsic；
5. 分析通信瓶颈，理解 ghost zone 交换量、同步开销、消息合并、通信与计算重叠、负载均衡。

9 要点速查

主题	要点	注意
并行现状	MPI 已用, OpenMP 未启用	OMP_threads 当前只是环境变量
关键文件	bssn_rhs.f90, diff_new.f90, Parallel.C 等	重点在 RHS 与通信
表面串行	block/patch/detector/grid level 间可并行	profile 后再动
fork-join	并行区域过小得不偿失	每线程至少几百微秒
MPI+OpenMP	每 NUMA 节点一个 rank, 内部多线程	乘积不超过总核数

NUMA 工具	lscpu, numactl --hardware, hwloc-ls	先看拓扑再绑
绑核	mpirun --bind-to, OMP_PROC_BIND/OMP_PLACES, numactl	避免跨 NUMA 远端访问
自动向量化	清掉分支/依赖/不连续/别名	是免费的午餐
SIMD intrinsic	AArch64 NEON/SVE	只对真正热点手写
ghost zone	通信量 $\propto 6n^2g$, 计算量 $\propto n^3$	rank 越多通信占比越高
消息合并	合并小消息降延迟占比	别一条条发
通信计算重叠	MPI_Isend/Irecv + 内部算 + MPI_Wait	掩盖通信时间
负载均衡	rank 工作量要接近	最慢 rank 拖死整体
配置参数	MPI_processes, OMP_threads	OMP_threads 需 CMake + 指令才生效

10 小结

ABE 的 baseline 只用了 MPI, OpenMP 虽然有环境变量但没真正生效, 这给我们留下了并行结构与向量化的双重空间。我们先把现状摸清: 关键文件是 `bssn_rhs.f90` 与 `Parallel.C/MPatch.C`, 并行结构优化的核心是重新平衡 MPI rank 与 OpenMP 线程, 并警惕 fork-join、false sharing 与跨 NUMA 远端访问。在 NUMA 层面, 先用 `lscpu`、`numactl --hardware`、`hwloc-ls` 看拓扑, 再用 `mpirun` 绑核参数、`OMP_PROC_BIND/OMP_PLACES`、`numactl` 把进程与内存绑在本地节点。计算 kernel 上, 先帮编译器自动向量化 (清分支、清依赖、清别名、让数据连续), 再对真正热点用 AArch64 NEON/SVE intrinsic。通信上, 理解 ghost zone 交换量随 rank 数增长, 合并小消息, 用非阻塞通信重叠通信与计算, 并保证 rank 间负载均衡。最后记住 `OMP_threads` 只有在 CMake 启用 OpenMP 且源码加了指令后才真正生效。下一讲我们将进入 ABEGPU 的 GPU 演化路径, 看 GPU kernel 怎么优化。

讲义基于 HPC101 2025 Lab4 实验内容编写